

Resumen Segundo Parcial — Programación I

Fecha del parcial: 26/06/2026 | Clases: 6 a 15

1. DICCIONARIOS (Clase 6)

Estructura de datos que almacena pares **clave:valor**. Son **mutables**, **no ordenados** y las claves son **únicas**.

Creación y acceso

```
alumno = {"nombre": "Oliver", "edad": 5}
```

```
alumno["edad"] = 6      # modificar
```

```
alumno["apellido"] = "Pérez" # agregar
```

```
print(alumno["nombre"])  # acceder → KeyError si no existe
```

Métodos clave

Método / operador	Qué hace
<code>dict.get(clave, default)</code>	Devuelve el valor o el default (sin KeyError)
<code>dict.keys()</code>	Iterador de claves
<code>dict.values()</code>	Iterador de valores
<code>dict.items()</code>	Iterador de tuplas (clave, valor)
<code>del dict[clave]</code>	Elimina el par
<code>dict.clear()</code>	Vacía el diccionario
<code>dict.update({...})</code>	Agrega o actualiza pares
<code>"clave" in dict</code>	Verifica existencia de clave
<code>len(dict)</code>	Cantidad de pares
<code>dict(sorted(d.items()))</code>	Ordenar por clave

Recorridos

```
for clave, valor in alumno.items():
```

```
    print(clave, "-->", valor)
```

Anidamiento y listas de diccionarios

```
# Diccionario anidado
```

```
pais = {"nombre": "Argentina", "coordenadas": {"latitud": -34, "longitud": -64}}
```

```
print(pais["coordenadas"]["latitud"]) # -34
```

```
# Lista de diccionarios → como una tabla (índice + clave)
```

```
ciudades = [{"cp": 1425, "nombre": "Capital"}, {"cp": 1900, "nombre": "La Plata"}]
```

```
print(ciudades[1]["nombre"]) # "La Plata"
```

JSON

```
import json
```

```
# Leer JSON
```

```
with open("paises.json", "r") as f:
```

```
    paises = json.loads(f.read())
```

```
# Escribir JSON
```

```
paisesJSON = json.dumps(paises, indent=4)
```

```
with open("paises.json", "w") as f:
```

```
    f.write(paisesJSON)
```

2. CONJUNTOS (Clase 6)

Colección **desordenada, sin duplicados, sin índices**. Equivalente a los conjuntos matemáticos.

Creación

```
fibonacci = {0, 1, 1, 2, 3} # → {0, 1, 2, 3} (duplicados eliminados)
```

```
fibonacci = set([0, 1, 1, 2]) # desde lista
```

```
vacio = set() # ⚠ {} crea un dict, no un set
```

Comprensión de conjuntos

```
cuadrados = {i**2 for i in [1, 2, 2, 3]} # {1, 4, 9}
```

Agregar / eliminar

```
s.add("elemento") # agrega uno
```

```
s.update(["a", "b"]) # agrega varios
```

```
s.remove("x") # KeyError si no existe
```

```
s.discard("x") # no lanza error si no existe
```

```
s.clear() # vacía el set
```

Operaciones matemáticas

Operación	Símbolo	Método
Unión		<code>.union()</code>
Intersección	&	<code>.intersection()</code>
Diferencia (A - B)	-	<code>.difference()</code>
Diferencia simétrica	^	<code>.symmetric_difference()</code>
Subconjunto	<=	<code>.issubset()</code>

`fibonacci = {0, 1, 2, 3, 5, 8, 13}`

`pares = {2, 4, 6, 8, 10}`

`fibonacci | pares` # unión: todos

`fibonacci & pares` # intersección: {8, 2}

`fibonacci - pares` # diferencia: {0, 1, 3, 5, 13}

`fibonacci ^ pares` # dif. simétrica: los que están en uno solo

Usos prácticos

Eliminar duplicados de una lista

`unicos = list(set(["a", "b", "a", "c"]))`

Verificar pertenencia

`if "elemento" in mi_set:`

`...`

Funciones de agregación

`len()`, `max()`, `min()`, `sum()` funcionan igual que con listas.

3. FUNCIONES LAMBDA, MAP, FILTER, REDUCE (Clase 9)

Lambda

Función anónima de una sola expresión.

`lambda p1, p2 : expresión`

`doble = lambda n: n * 2`

`impar = lambda n: n % 2 != 0`

`mayor = lambda x, y: x if x > y else y` # con condicional

Usar `lambda` para cosas simples y de corto uso. Para lógica compleja → `def`.

Map

Aplica una función a **cada elemento** de un iterable.

```
resultado = list(map(lambda n: n**2, [1, 2, 4, 7])) # [1, 4, 16, 49]
```

```
resultado = tuple(map(lambda n: n**2, (1, 2, 4))) # tupla
```

Filter

Retorna los elementos que cumplen una condición (`True`).

```
pares = list(filter(lambda n: n % 2 == 0, [1, 2, 3, 4, 5, 6])) # [2, 4, 6]
```

Reduce

"Reduce" un iterable a **un único valor** acumulando de izquierda a derecha.

```
from functools import reduce
```

```
total = reduce(lambda x, y: x + y, [1, 2, 3, 4]) # 10
```

```
maximo = reduce(lambda x, y: x if x > y else y, [3, 1, 9, 2]) # 9
```

4. EXCEPCIONES (Clase 10)

Tipos de error

- **Sintaxis** (`SyntaxError`): el código no se puede leer/interpretar.
- **Semántica**: el código corre pero hace algo incorrecto (lógica equivocada).
- **Ejecución**: ocurre mientras el programa corre.

Excepciones comunes

Excepción	Cuándo ocurre
<code>ZeroDivisionError</code>	División por cero
<code>TypeError</code>	Operación con tipo incorrecto

Excepción	Cuándo ocurre
<code>IndexError</code>	Índice fuera de rango
<code>KeyError</code>	Clave inexistente en dict
<code>OverflowError</code>	Número demasiado grande
<code>ValueError</code>	Valor de tipo correcto pero inválido
<code>FileNotFoundError</code>	Archivo no encontrado

Estructura try/except

try:

```
    resultado = 10 / 0
```

except ZeroDivisionError:

```
    print("No se puede dividir por cero")
```

except TypeError:

```
    print("Tipo incorrecto")
```

except Exception as e: # captura cualquier excepción

```
    print(f"Error: {e}")
```

```
    print(f"Tipo: {type(e).__name__}")
```

else:

```
    print("Todo salió bien") # solo si NO hubo excepción
```

finally:

```
    print("Esto siempre se ejecuta") # limpieza
```

Raise

Lanzar una excepción de forma manual:

```
def raizCuadrada(n):  
  
    if n < 0:  
  
        raise ValueError("No puede ser negativo")  
  
    return n ** 0.5
```

Excepciones personalizadas

```
class SaldoInsuficienteError(Exception):  
  
    def __init__(self, mensaje):  
  
        super().__init__(mensaje)  
  
raise SaldoInsuficienteError("Fondos insuficientes")
```

5. TESTS UNITARIOS (Clase 11)

Entornos

Desarrollo → Pruebas → Pre-producción → Producción

Assert

assert expresion, "mensaje de error si falla"

assert suma(2, 2) == 4, "2+2 debería ser 4"

Pytest

Framework de testing. Instalar: `pip install pytest`

Convenciones de nombres (obligatorias para que pytest las detecte):

- Archivos: `test_*.py` o `*_test.py`
- Funciones: deben comenzar con `test_`

calculadora.py

```
def dividir(a, b):
```

```
if b == 0:

    raise ValueError("No se puede dividir por cero")

return a / b

# test_calculadora.py

import pytest

from calculadora import dividir

def test_division_positiva():

    assert dividir(10, 2) == 5

def test_division_por_cero():

    with pytest.raises(ValueError):

        dividir(10, 0)
```

Ejecutar:

```
pytest      # básico

pytest -v   # verbose (muestra cada test)
```

Buenas prácticas:

- Probar el camino feliz (caso normal).
 - Probar casos borde (lista vacía, None, tipos incorrectos).
 - Probar que las excepciones se lanzan cuando corresponde.
-

6. ARCHIVOS (Clase 12)

Modos de apertura

Modo	Acción
"r"	Solo lectura
"w"	Solo escritura (sobreescribe o crea)
"a"	Agregar al final (o crea si no existe)

Leer

try:

```
contenido = open("archivo.txt", "r", encoding="UTF-8")
```

```
for linea in contenido:
```

```
    print(linea)
```

```
contenido.close()
```

except FileNotFoundError:

```
    print("No existe el archivo")
```

Leer todo de una vez

```
lineas = contenido.read()      # string completo
```

```
renglones = lineas.split("\n") # lista de líneas
```

Escribir

```
archivo = open("personas.txt", "w")
```

```
archivo.write("Oliver\n")
```

```
archivo.close()
```

Con print (agrega \n automáticamente)

```
print("Oliver", file=archivo)
```

With open (recomendado)

Cierra el archivo automáticamente, incluso si hay error:

```
with open("registro.txt", "a") as archivo:
```

```
    archivo.write("Nueva línea\n")
```

JSON con archivos

```
import json
```

Leer

```
with open("paises.json", "r") as f:
```

```
    paises = json.loads(f.read())
```

Escribir

```
with open("paises.json", "w") as f:
```

```
    f.write(json.dumps(paises, indent=4))
```

7. EXPRESIONES REGULARES (Clase 13)

Patrones para buscar/validar texto. Se importa el módulo `re`.

```
import re
```

Metacaracteres

Símbolo	Significado
.	Cualquier carácter (excepto \n)
^	Inicio de cadena (o negación dentro de [])

Símbolo	Significado
\$	Fin de cadena
\	Escape de metacaracter
[abc]	Clase: a, b o c
[^abc]	Negación: todo menos a, b, c
[a-z]	Rango
	Alternativa (o lógico)

Cuantificadores

Símbolo	Significado
?	0 o 1 vez
*	0 o más veces
+	1 o más veces
{n}	exactamente n veces
{n,m}	entre n y m veces
(...)	Agrupación / captura

Métodos del módulo **re**

match: verifica desde el INICIO de la cadena

```
re.match(patron, cadena)
```

search: busca en CUALQUIER posición, devuelve el primero

```
match = re.search('[0-9]+', "precio: $11.33")
```

```
match.group() # valor encontrado
```

```
match.start() # posición inicio
```

```
match.end() # posición fin
```

`match.span()` # tupla (inicio, fin)

findall: todas las coincidencias como lista

`re.findall(patron, texto)`

finditer: todas las coincidencias como iterador de objetos Match

`for m in re.finditer(patron, texto):`

`print(m.group(), m.start())`

sub: reemplazar

`re.sub(patron, reemplazo, texto)`

split: dividir por patrón

`re.split('[.,? ]+', texto)`

compile: compilar para reutilizar

`patron = re.compile('[0-9]{4}')`

`patron.findall(cadena)`

`patron.sub('XXXX', cadena)`

Ignorar mayúsculas: `re.search(patron, cadena, re.IGNORECASE)`

Ejemplos de patrones

`[A-Za-z][1-3]{3}` # letra seguida de 3 dígitos del 1 al 3

`[0-9]{4}` # año de 4 dígitos

`[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}` # email

`^(0?[1-9]|[12][0-9]|3[01])/(0?[1-9]|1[0-2])/[0-9]{4}$` # fecha DD/MM/AAAA

8. RECURSIVIDAD (Clase 14)

Una función que **se llama a sí misma** para resolver subproblemas más chicos.

Requisitos:

1. **Caso base:** condición de parada (evita el bucle infinito).
2. **Llamada recursiva:** con parámetros que se acercan al caso base.

Sin retorno

```
def cuentaRegresiva(n):
```

```
    n -= 1
```

```
    if n > 0:
```

```
        print(n)
```

```
        cuentaRegresiva(n) # llamada recursiva
```

```
    else:
```

```
        print("Fin")    # caso base
```

Con retorno (factorial)

```
def factorial(n):
```

```
    if n == 1:    # caso base
```

```
        return 1
```

```
    return n * factorial(n - 1) # llamada recursiva
```

```
# factorial(3) = 3 * factorial(2) = 3 * 2 * factorial(1) = 3 * 2 * 1 = 6
```

Fibonacci

```
def fibonacci(n):
```

```
    if n == 0 or n == 1: # caso base
```

```
    return n
```

```
    return fibonacci(n-1) + fibonacci(n-2)
```

⚠ Ineficiente para n grande: repite cálculos. Solución → **memoization**.

Memoization

Guardar resultados calculados para no repetirlos:

```
cache = {}
```

```
def fibonacci(n):
```

```
    if n in cache:
```

```
        return cache[n]
```

```
    if n == 0 or n == 1:
```

```
        resultado = n
```

```
    else:
```

```
        resultado = fibonacci(n-1) + fibonacci(n-2)
```

```
    cache[n] = resultado
```

```
    return resultado
```

Límite de recursión

Python tiene un límite de ~1000 llamadas recursivas. Se puede modificar:

```
import sys
```

```
sys.getrecursionlimit() # ver límite actual
```

```
sys.setrecursionlimit(2000) # cambiar límite
```

Superar el límite lanza **RecursionError**.

Conversión decimal a binario

```
def decimalABinario(n):
```

```
    if n == 0: return "0"
```

```
    elif n == 1: return "1"
```

```
    return decimalABinario(n // 2) + str(n % 2)
```

Tabla resumen: ¿qué entra en cada clase?

Clase	Fecha	Tema
6	17/04	Diccionarios y Conjuntos
9	08/05	Lambda, map, filter, reduce
10	15/05	Excepciones
11	22/05	Tests unitarios (pytest)
12	29/05	Archivos (txt, csv, json)
13	05/06	Expresiones Regulares
14	12/06	Recursividad
15	19/06	Repaso y modelo